College Data Science Lab Exam Study Guide

Exercise 9: Linear Regression Algorithm (Simple Linear Regression)

Source File: `e09_linear_regression.py` • College of Engineering Trivandrum (CET) • 20MCA241 Data Science Lab

---

Table of Contents

1. Program Identification and Aim
2. Prerequisites and Conceptual Framework (Simple vs Multiple Regression)
3. Complete Exam-Ready Algorithm (START → STOP Format)
4. Complete Mathematical Theory (OLS, Slope, Intercept, MSE, R^2)
5. Complete Original Source Code & Detailed Walkthrough
6. Complete Function, Parameter, and Syntax Reference
7. Step-by-Step Execution Walkthrough & Data Transformations
8. Output, Regression Line Plot & Metric Interpretation
9. Exam Preparation: Code Skeleton & Writing Strategy
10. Comprehensive Viva Voce Questions & Answers (16 Detailed Q&As)
11. Common Mistakes, Traps, and Troubleshooting
12. Memory Tricks & Last-Minute Rapid Revision Sheet
13. Cross-Program Master Comparison Table (Programs 05 to 10)

Section 1: Program Identification and Aim

Experiment Number: Exercise 9

Script Filename: `e09_linear_regression.py`

Course: 20MCA241 Data Science Lab • Semester 3 MCA

Institution: Department of Computer Applications, College of Engineering Trivandrum (CET)

> **Official Lab Manual Aim**
>
> *Program to implement Linear Regression Algorithm using any standard dataset available in the public domain.*

Dataset Used: Scikit-learn's standard Diabetes Dataset (442 samples × 10 physiological baseline variables). Per college manual specification, exactly **one independent feature** is extracted: **BMI (Body Mass Index, feature index 2)**.

Dependent Target (y): Quantitative measure of disease progression one year after baseline.

Data Partitioning: Slicing split leaving the last 20 samples for model testing (422 samples for training, 20 samples for evaluation).

Section 2: Prerequisites and Conceptual Framework

Linear Regression models the relationship between a scalar dependent response variable Y and one or more explanatory explanatory variables X:

- **Simple Linear Regression (Program 9):** Involves exactly **one independent variable** ($X = \text{BMI}$). The model fits a straight 2D line: $\hat{Y} = b_0 + b_1 X$.
- **Multiple Linear Regression (Program 10):** Involves **two or more independent variables** ($X_1 = \text{Age}$, $X_2 = \text{BMI}$). The model fits a multi-dimensional hyperplane: $\hat{Y} = b_0 + b_1 X_1 + b_2 X_2$.
- **The 2D Matrix Requirement in scikit-learn:** Scikit-learn estimators require feature matrix X to have shape `(n_samples, n_features)`. When selecting a single feature, slicing `diabetes_X[:, 2]` produces a 1D vector of shape `(442,)` which causes an error. Using `np.newaxis` (i.e. `diabetes_X[:, np.newaxis, 2]`) adds a new axis, creating a valid 2D array of shape `(442, 1)`.

Section 3: Complete Exam-Ready Algorithm

This algorithm follows the exact wording and structure from the official CET lab manual (pages 24–25), formatted in the standard numbered START → STOP layout for lab record books and exam answer sheets.

GENERAL ALGORITHM: LINEAR REGRESSION

Step 1 [START]: Begin the program execution.
Step 2 [Import Libraries]: Import `matplotlib.pyplot`, `numpy`, `datasets`, `linear_model`, and metrics (`mean_squared_error`, `r2_score`) from `sklearn`.
Step 3 [Load Dataset]: Load the diabetes dataset directly using `datasets.load_diabetes(return_X_y=True)` to retrieve feature matrix `diabetes_X` and target vector `diabetes_y`.
Step 4 [Feature Selection (Single Feature)]: Extract only the BMI feature (index 2) as a 2D column vector using `diabetes_X = diabetes_X[:, np.newaxis, 2]`.
Step 5 [Partition Dataset]:

- Split features into training set `diabetes_X_train = diabetes_X[:-20]` and testing set `diabetes_X_test = diabetes_X[-20:]`.
- Split targets into training set `diabetes_y_train = diabetes_y[:-20]` and testing set `diabetes_y_test = diabetes_y[-20:]`.

Step 6 [Initialize & Fit Model]: Create linear regression object `regr = linear_model.LinearRegression()` and fit it on training data: `regr.fit(diabetes_X_train, diabetes_y_train)`.
Step 7 [Predict on Test Data]: Generate predicted target values on test features: `diabetes_y_pred = regr.predict(diabetes_X_test)`.
Step 8 [Compute & Display Metrics]:

- Display regression slope coefficient: `regr.coef_`.
- Display regression intercept: `regr.intercept_`.
- Calculate and print Mean Squared Error: `mean_squared_error(diabetes_y_test, diabetes_y_pred)`.
- Calculate and print Coefficient of Determination (R^2): `r2_score(diabetes_y_test, diabetes_y_pred)`.

Step 9 [Plot Regression Line]:

- Scatter actual test data points in black: `plt.scatter(diabetes_X_test, diabetes_y_test, color="black")`.
- Sort test feature indices using `sorted_idx = np.argsort(diabetes_X_test[:, 0])` to guarantee a smooth continuous line.
- Plot the fitted regression line in blue: `plt.plot(diabetes_X_test[sorted_idx], diabetes_y_pred[sorted_idx], color="blue", linewidth=3)`.
- Set labels, title, tick parameters, and display via `plt.show()`.

Step 10 [STOP]: Terminate program execution.

Section 4: Complete Mathematical Theory

1. Simple Linear Regression Model Equation

The relationship between the dependent variable Y and independent variable X is modeled as:

$$\hat{Y} = b_0 + b_1 X$$ Where:

- $\hat{Y}$: Predicted target value (disease progression).
- X: Input feature (BMI).
- b_1: Slope coefficient (rate of change of Y per unit change in X).
- b_0: Y-intercept (value of $\hat{Y}$ when $X = 0$).

2. Closed-Form Ordinary Least Squares (OLS) Formulas

Given n observations $(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)$ with sample means $\bar{x} = \frac{\sum x_i}{n}$ and $\bar{y} = \frac{\sum y_i}{n}$:

$$b_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)}$$ $$b_0 = \bar{y} - b_1 \bar{x}$$

3. Mean Squared Error (MSE)

Measures the average squared difference between true observations y_i and predicted values $\hat{y}_i$:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$ A smaller MSE indicates closer alignment between predictions and actual values.

4. Coefficient of Determination (R^2 Score)

Proportion of the variance in the dependent variable that is predictable from the independent variable:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- $R^2 = 1.0$: Perfect predictions ($\text{MSE} = 0$).
- $R^2 = 0.0$: Model performs no better than simply predicting the mean $\bar{y}$.
- $R^2 < 0.0$: Model performs worse than the horizontal mean line (possible on test sets).

Section 5: Complete Original Source Code & Detailed Walkthrough

Below is the complete, tested source code of `e09_linear_regression.py` matching the college manual:

```
# Exercise 9: Linear Regression

# Aim: Program to implement Linear Regression Algorithm using any standard dataset available in the public domain.

import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load the diabetes dataset
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)

# 2. Use only one feature (BMI is feature index 2)
diabetes_X = diabetes_X[:, np.newaxis, 2]

# 3. Split the features and targets into training and testing sets (last 20 samples for testing)
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]

diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

# 4. Create and train linear regression model
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)

# 5. Make predictions using the test set
diabetes_y_pred = regr.predict(diabetes_X_test)

# 6. Print evaluation metrics
print("Coefficients: ", regr.coef_)
print("Intercept: ", regr.intercept_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test, diabetes_y_pred))
print("R^2 score: %.2f" % r2_score(diabetes_y_test, diabetes_y_pred))

# 7. Plot test data points and the fitted regression line
plt.scatter(diabetes_X_test, diabetes_y_test, color="black")

sorted_idx = np.argsort(diabetes_X_test[:, 0])
plt.plot(diabetes_X_test[sorted_idx], diabetes_y_pred[sorted_idx], color="blue", linewidth=3)

plt.xlabel("BMI")
plt.ylabel("Diabetes progression")
plt.title("Linear Regression: BMI vs Diabetes Progression")
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.show()
```

Line-by-Line Code Walkthrough

Line(s)	Code Statement	Operational Purpose & Beginner Explanation
5-8	`import matplotlib.pyplot as plt ...`	Imports plotting tools, numpy array utilities, sklearn dataset and linear_model modules, and evaluation metrics.
11	`datasets.load_diabetes(return_X_y=True)`	Loads the diabetes dataset directly as a tuple `(X, y)` rather than a Bunch dictionary object.
14	`diabetes_X[:, np.newaxis, 2]`	Extracts feature column index 2 (BMI) and reshapes it from 1D $(442,)$ to 2D $(442, 1)$ via `np.newaxis`.
17-21	`diabetes_X[:-20], diabetes_X[-20:] ...`	Splits first 422 rows into training sets and final 20 rows into testing sets using negative slice indexing.
24	`regr = linear_model.LinearRegression()`	Instantiates the Ordinary Least Squares (OLS) linear regression model.
25	`regr.fit(diabetes_X_train, diabetes_y_train)`	Calculates optimal slope b_1 and intercept b_0 by minimizing squared residuals on the 422 training samples.
28	`diabetes_y_pred = regr.predict(diabetes_X_test)`	Evaluates $\hat{y} = b_0 + b_1 x$ for the 20 unseen test instances.
31-34	`print("Coefficients:", regr.coef_) ...`	Outputs slope (938.24), intercept (152.92), MSE (2548.07), and R^2 (0.47) formatted to 2 decimals.
37	`plt.scatter(diabetes_X_test, ...)`	Plots the 20 actual test points as black scatter dots on the Cartesian coordinate plane.
39-40	`sorted_idx = np.argsort(...)`	Sorts test X coordinates in ascending order so `plt.plot()` draws a smooth continuous line rather than a back-and-forth zigzag.
42-47	`plt.xlabel("BMI"); plt.show()`	Applies axis labels, plot title, font size specifications, and displays the regression graph.

Section 6: Complete Function, Parameter, and Syntax Reference

Function / Attribute	Library	Key Parameters / Return Value	Exam Role & Significance
`load_diabetes(return_X_y=True)`	`sklearn.datasets`	`return_X_y=True` returns tuple `(X, y)` directly.	Fast, clean dataset loading without navigating bunch attributes.
`np.newaxis`	`numpy`	Inserts a new dimension of size 1 into the sliced array.	Transforms 1D vector shape $(442,)$ into 2D matrix shape $(442, 1)$ required by scikit-learn.
`[:-20]` and `[-20:]`	Python slice notation	`[:-20]` selects all elements except the last 20; `[-20:]` selects the last 20.	Simple deterministic train/test splitting without requiring `train_test_split()`.
`LinearRegression()`	`sklearn.linear_model`	Ordinary Least Squares linear regression.	Core regression estimator.
`regr.coef_`	`LinearRegression`	Array of estimated slope coefficients (b_1).	For simple regression, contains 1 float: $[938.23786125]$.
`regr.intercept_`	`LinearRegression`	Estimated intercept float (b_0).	Baseline expected value when $X=0$: 152.91886182616113.
`np.argsort()`	`numpy`	Returns indices that would sort the 1D array in ascending order.	Ensures that `plt.plot()` connects points monotonically along the x-axis.

Section 7: Step-by-Step Execution Walkthrough

Step	Code Line	Array / Variable	Shape & Type	Contents / Value Description
1	11	`diabetes_X, diabetes_y`	`(442, 10)` and `(442,)`	Full raw dataset: 442 patients, 10 normalized features.
2	14	`diabetes_X`	`ndarray (442, 1)`, float64	Single BMI column converted into a 2D matrix via `np.newaxis`.
3	17-21	`diabetes_X_train, diabetes_X_test`	`(422, 1)` and `(20, 1)`	Training partition (first 422) and test partition (last 20).
4	25	`regr`	Fitted Estimator	Learned parameters: $b_1 = 938.24$, $b_0 = 152.92$.
5	28	`diabetes_y_pred`	`ndarray (20,)`, float64	Model predictions for the 20 test patients.
6	33-34	MSE and R^2	Python floats	$\text{MSE} = 2548.07$; $R^2 = 0.47$.

Section 8: Output and Interpretation

1. Console Terminal Output

```
Coefficients:  [938.23786125]
Intercept:  152.91886182616113
Mean squared error: 2548.07
R^2 score: 0.47
```

(These numbers match the physical manual screenshot on page 24).

2. Mathematical Interpretation of Results

- **Slope ($b_1 = 938.24$):** Positive coefficient indicates that higher BMI strongly correlates with increased diabetes progression. Specifically, for each 0.01 unit increase in normalized BMI, diabetes progression increases by approximately 9.38 points.
- **Intercept ($b_0 = 152.92$):** The expected baseline diabetes progression score for an average individual with centered zero BMI.
- **Mean Squared Error ($\text{MSE} = 2548.07$):** Root Mean Squared Error ($\text{RMSE} = \sqrt{2548.07} \approx 50.48$). The model's predictions deviate from actual progression values by roughly 50 points on average.
- **R^2 Score (0.47):** Approximately 47% of the variance in diabetes progression across the test cohort is accounted for by BMI alone. This confirms BMI is a potent single predictor.

Section 9: Exam Preparation: Code Skeleton & Writing Strategy

5-Minute Exam Memorization Skeleton

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

X, y = datasets.load_diabetes(return_X_y=True)
X = X[:, np.newaxis, 2]

X_train, X_test = X[:-20], X[-20:]
y_train, y_test = y[:-20], y[-20:]

regr = linear_model.LinearRegression().fit(X_train, y_train)
y_pred = regr.predict(X_test)

print("Coefficients:", regr.coef_)
print("Intercept:", regr.intercept_)
print("MSE: %.2f" % mean_squared_error(y_test, y_pred))
print("R^2: %.2f" % r2_score(y_test, y_pred))

plt.scatter(X_test, y_test, color="black")
idx = np.argsort(X_test[:, 0])
plt.plot(X_test[idx], y_pred[idx], color="blue", linewidth=3)
plt.xlabel("BMI"); plt.ylabel("Progression"); plt.show()
```

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Q1. What is the difference between Simple Linear Regression and Multiple Linear Regression?

Answer: Simple Linear Regression uses exactly one independent explanatory variable to predict the target ($\hat{y} = b_0 + b_1 x$), forming a 2D line. Multiple Linear Regression uses two or more explanatory variables ($\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + ... + b_n x_n$), forming a multi-dimensional hyperplane.

Q2. Why is `np.newaxis` used when slicing `diabetes_X[:, np.newaxis, 2]`?

Answer: In NumPy, slicing a single column `diabetes_X[:, 2]` yields a 1D array of shape $(442,)$. However, scikit-learn's `fit()` method mandates a 2D feature matrix of shape $(N, \text{features})$. `np.newaxis` inserts an extra dimension, converting the shape into $(442, 1)$.

Q3. Why is `np.argsort()` applied prior to calling `plt.plot()`?

Answer: `plt.plot()` connects data points sequentially in the exact order they appear in the array. Since test feature values are not sorted, plotting unsorted points produces a tangled, back-and-forth zigzag line. `np.argsort()` provides sorted indices along the x-axis, rendering a clean, monotonic regression line.

Q4. What does the Diabetes dataset contain?

Answer: It comprises 442 diabetes patients with 10 baseline demographic and physiological features (Age, Sex, BMI, Average Blood Pressure, and six blood serum measurements) that have been mean-centered and scaled by the standard deviation times the square root of n. The target is a quantitative disease progression score.

Q5. What mathematical method does `LinearRegression()` use to solve for coefficients?

Answer: It uses the **Ordinary Least Squares (OLS)** closed-form analytical solution via Singular Value Decomposition (SVD) of X (equivalent to the normal equation $\hat{\beta} = (X^T X)^{-1} X^T Y$), which directly minimizes the sum of squared residuals without requiring iterative gradient descent.

Q6. What does a slope coefficient of 938.24 signify in this model?

Answer: It represents the rate of change: for each 1-unit increase in standardized BMI, the predicted disease progression increases by 938.24 units. It demonstrates a positive linear relationship between body mass index and diabetes severity.

Q7. What is the R^2 score and how is it interpreted?

Answer: R^2 is the coefficient of determination, calculated as $1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$. It quantifies the proportion of target variance explained by the model. Here, $R^2 = 0.47$ indicates that 47% of the variability in disease progression is explained by BMI.

Q8. Can the R^2 score ever be negative?

Answer: Yes. On an unseen test dataset, if the model's predictions are worse than simply predicting the mean of the test labels ($\bar{y}_{\text{test}}$), $SS_{\text{res}} > SS_{\text{tot}}$, causing R^2 to become negative.

Q9. What is Mean Squared Error (MSE) and what is its primary drawback?

Answer: MSE is the average of squared prediction errors $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$. Its main drawback is that it is expressed in squared units of the target variable (2548.07 points^2), making direct physical interpretation difficult. Taking its square root (RMSE ≈ 50.48) restores the original units.

Q10. How does the train/test split `[:-20]` and `[-20:]` work?

Answer: It uses Python negative slicing. `[:-20]` selects all samples from index 0 up to (but excluding) the 20th sample from the end ($442 - 20 = 422$ samples for training). `[-20:]` selects the final 20 samples (from index 422 to 441) for testing.

Q11. What are the key statistical assumptions of Ordinary Least Squares regression?

Answer:

1. **Linearity:** The relationship between independent features and target is linear.
2. **Homoscedasticity:** Residual variance is constant across all values of X.
3. **Independence:** Errors are statistically independent with no autocorrelation.
4. **Normality:** Residual errors are normally distributed around zero mean.

Q12. What is a residual?

Answer: A residual is the difference between an observed true target value and its corresponding model prediction: $e_i = y_i - \hat{y}_i$.

Q13. Why is linear regression sensitive to outliers?

Answer: Because OLS minimizes *squared* residuals ($\sum e_i^2$). A point far away from the regression line exerts disproportionate leverage, shifting the slope and intercept toward itself to reduce that large squared error penalty.

Q14. What does `regr.intercept_` represent geometrically?

Answer: It is the value at which the fitted regression line intersects the vertical y-axis (the expected output value when the input feature $X = 0$). Here it equals 152.92.

Q15. How do you predict progression for an individual patient with $\text{BMI} = 0.05$?

Answer: Pass a 2D array to predict: `regr.predict([[0.05]])`, which computes $152.92 + 938.24 \times 0.05 = 199.83$.

Q16. What is the time complexity of Simple Linear Regression?

Answer: Training is $\mathcal{O}(n)$, where n is the number of samples (422), because computing means and sums of products requires a single linear pass over the data. Prediction is $\mathcal{O}(1)$ per sample.

Section 11: Common Mistakes, Traps, and Troubleshooting

Common Mistake	Error Message / Negative Effect	Correct Approach
Omitting `np.newaxis`: `diabetes_X[:, 2]`	`ValueError: Expected 2D array, got 1D array instead` during `regr.fit()`.	Use `diabetes_X[:, np.newaxis, 2]` or `.reshape(-1, 1)`.
Forgetting `np.argsort()` before plotting	Matplotlib connects points in random order, creating a tangled, overlapping zig-zag spiderweb across the plot.	Always sort indices: `sorted_idx = np.argsort(diabetes_X_test[:, 0])` and plot `diabetes_X_test[sorted_idx]`.
Mixing up `return_X_y=True`	Assigning to a single variable: `diabetes = datasets.load_diabetes()` then attempting `diabetes[:, np.newaxis, 2]` fails because `Bunch` is a dict, not an array.	Use `diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)`.
Evaluating training set instead of test set	Passing training data to `mean_squared_error(diabetes_y_train, diabetes_y_pred)` causes shape mismatch or gives in-sample error.	Always evaluate test predictions: `mean_squared_error(diabetes_y_test, diabetes_y_pred)`.

Section 12: Memory Tricks & Last-Minute Rapid Revision Sheet

Rapid Formula Sheet

- **Dataset Loader:** `X, y = datasets.load_diabetes(return_X_y=True)`.
- **Single Feature Reshape:** `X = X[:, np.newaxis, 2]` (BMI).
- **Train/Test Partition:** `[:-20]` for training, `[-20:]` for testing.
- **Model:** `regr = linear_model.LinearRegression().fit(X_train, y_train)`.
- **Metrics:** `regr.coef_` (slope), `regr.intercept_` (intercept), `mean_squared_error()`, `r2_score()`.
- **Line Plotting:** `plt.plot(X_test[sorted_idx], y_pred[sorted_idx], color="blue")` after `np.argsort()`.

Section 13: Cross-Program Master Comparison Table (Programs 05 to 10)

This master comparison summarizes all Machine Learning experiments in the college Data Science curriculum:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	Structured text classification report
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"` , `max_depth=4` , `min_samples_leaf=2`)	`classification_report()`	Flowchart tree via `tree.plot_tree(filled=True)`
e08_kmeans.py	**Unsupervised Clustering**	Iris Dataset (First 2 Features: 150 × 2)	None (Full unlabeled dataset X)	K-Means Clustering (`n_clusters=3` , `random_state=21`)	Adjusted Rand Index (ARI) & Normalized Mutual Info (NMI)	2 Subplots: Actual (`prism`) vs Predicted (`jet`)
e09_linear_regression.py	**Supervised Regression (Simple)**	Diabetes Dataset (1 Feature: BMI, Index 2)	Slicing (`[:-20]` train, `[-20:]` test)	Ordinary Least Squares (`LinearRegression()`)	Coefficients, Intercept, MSE, R^2 Score	Scatter plot of test samples with fitted linear line
e10_multiple_linear_regression.py	**Supervised Regression (Multiple)**	Diabetes Dataset (2 Features: Age & BMI, Indices 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Coefficients (vector of 2), Intercept, MSE, R^2 Score	Figure 1: Feature 0 & 2 subplots; Figure 2: Actual vs Predicted fit line